

Frontend — operator dashboard

Owner: *TBD*

Depends on: `docs/PROTOCOL.md`, `backend/`

WebSocket endpoint

Deliverable for M1: a public URL that a judge can open on their phone and understand in 20 seconds.

You own the demo. Everything else on this project is invisible to the panel; this is not.

1. Stack

Concern	Choice	Why
Build	Vite + React 18 + TypeScript	fast, zero config fights
Map	MapLibre GL JS + free raster tiles	no Mapbox token, no billing surprise mid-demo
Charts	Recharts	sparkline + timeline, minimal API

Concern	Choice	Why
State	Zustand	WebSocket stream into one store, no Redux ceremony
Styling	Tailwind	consistent under time pressure
Deploy	Vercel	push to main, get a URL

Do not add a component library that needs theming work. There is no time.

2. Setup

```
cd frontend
npm install
cp .env.example .env      # set
VITE_API_URL and VITE_WS_URL
npm run dev
```

`.env.example`

```
VITE_API_URL=http://localhost:8000
VITE_WS_URL=ws://localhost:8000/stream
```

3. Structure

```

frontend/src/
├─ api/
│   ├─ client.ts          REST wrapper
│   └─ socket.ts          WebSocket,
auto-reconnect with backoff
├─ store/
│   └─ feederStore.ts     nodes, events,
connection status
├─ components/
│   ├─ FeederMap.tsx      MapLibre, pole
pins, span polylines
│   ├─ NodeCard.tsx       sparkline,
battery, RSSI, state badge
│   ├─ NodePanel.tsx      scrollable list
of NodeCards
│   ├─ ScenarioPanel.tsx  the demo
buttons
│   ├─ CascadeOverlay.tsx break animation
+ latency counter
│   ├─ EventTimeline.tsx  chronological
event log with vote trail
│   ├─ CrewAlertMock.tsx  phone frame
showing the lineman's push
│   └─ StatusBar.tsx      feeder mode,
node count, live/offline
├─ types/protocol.ts      generated from
docs/PROTOCOL.md – mirror it exactly
└─ App.tsx

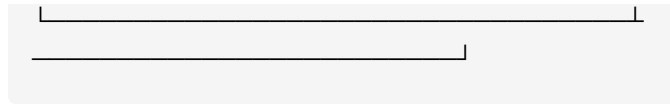
```

4. Layout

```

|
|
|   StatusBar   KSEB-TVM-F12 · 12 nodes ·
ALERT_ONLY · • LIVE |
|
|
|
|   ScenarioPanel
|       FeederMap
Break mid-feeder ] |
|   • — • — • — X — ○ — ○ — ○
Rain burst      ] |
|
|
Vegetation      ] |
|   green = normal
Substation outage ] |
|   amber = suspect
Node offline     ] |
|   red  = confirmed
Reset            ] |
|   grey = offline
|
|
|   NodePanel
|
|
N-006 ████ 4.9 100% |
|   EventTimeline
|
007 ████ 0.1 98% |
|   14:22:31 BREAK N-006 ↔ N-007 787ms |
008 ████ 0.1 97% |
|   14:19:02 FALSE POSITIVE REJECTED |
|

```



Mobile: map on top, scenario buttons collapse into a bottom sheet. Judges will open this on a phone – check it.

5. Task list

Phase 1 – skeleton (day 1)

- [] Vite + TS + Tailwind scaffold, deployed to Vercel on day one so the URL exists early
- [] `types/protocol.ts` transcribed from `PROTOCOL.md`, exact field names
- [] Mock JSON fixtures in `src/mocks/` so you can build before the backend is up
- [] StatusBar with a fake connection indicator

Phase 2 – map (days 2–3)

- [] MapLibre with OSM raster tiles, centred on the demo feeder
- [] Load feeder polyline + node coordinates from `GET /feeders`
- [] Pole pins coloured by `state`, span polylines coloured by the downstream node's state
- [] Click a pin → NodeCard opens

- [] Fit-bounds on load; no manual panning needed to see the whole feeder

Phase 3 — live data (days 4–5)

- [] `socket.ts` connects to `/stream`, reconnects with exponential backoff
- [] Zustand store ingests telemetry, keeps last 60 samples per node for sparklines
- [] NodeCard: E-field sparkline, baseline as a dashed reference line, battery bar, RSSI, state badge
- [] OFFLINE detection client-side too (no telemetry > 30 s) so the UI degrades honestly

Phase 4 — the demo mechanics (days 6–8)

- [] ScenarioPanel buttons → `POST /simulate/{scenario}`, disabled while a scenario runs
- [] **CascadeOverlay**: on a break, animate in sequence — node goes amber, neighbour votes fly along the line, quorum reached, span turns red, big millisecond counter lands on the final `latency_ms`
- [] **RECOVERED path**: rain and vegetation scenarios must visibly go amber and return to green, with a `FALSE POSITIVE REJECTED` line in

the timeline. Give this the same visual weight as a detection.

- [] EventTimeline with expandable vote trail per event
- [] CrewAlertMock: phone frame, push notification with span, GPS and a map thumbnail

Phase 5 — polish (day 9)

- [] Loading and error states; never a blank screen if the backend is cold-starting
- [] Latency badge on every event row
- [] "Simulated feeder · live consensus engine" label, always visible, not hidden in a tooltip
- [] Responsive check on a real phone
- [] Lighthouse pass — a slow dashboard reads as an unfinished dashboard

Phase 6 — v2, after the demo is submitted

- [] Crew PWA: alert → accept → navigate → mark restored
- [] Node commissioning flow (QR scan, assign to pole, set GPS)
- [] Per-feeder threshold tuning UI writing to `ArbiterConfig`
- [] Historical analytics: fault frequency by span, battery degradation trend

- [] Audit log view with actor and timestamp

6. Acceptance criteria for M1

1. Public URL loads in under 3 s on mobile data.
2. A stranger with no explanation can tell which poles are healthy.
3. Pressing "Break mid-feeder" produces a legible cascade ending in a latency number under 2000 ms.
4. Pressing "Rain burst" produces a visible amber-then-green recovery and a rejection line in the timeline.
5. Backend restart does not require a page refresh — the socket reconnects on its own.

7. Gotchas

- **Do not invent fields.** If you need something not in PROTOCOL.md, ask for a contract change; do not add it client-side. A silent divergence here costs a full day at integration.
- **Free Render/Railway instances cold-start.** Show a "connecting to feeder..." state, and ping the backend from the frontend on page load so it wakes before the judge presses a button.
- **Animation timing is not latency.** The cascade animation may take 3 s to play for legibility; the

displayed `latency_ms` must be the real number from the backend. Never animate the counter to a fake value.

- **Colour alone is not enough.** Add state text to badges — some judges will be colour-blind and the projector will wash out amber.
- Record a 90-second screen capture as a fallback. Venue wifi fails.